1주차 4일차: 입력 → 실행 → 결과 완성 — 120분

1. 학습 목표 — 5분

- 앞서 만든 입력창과 이벤트 처리기를 하나의 동작 흐름으로 이해합니다.
- 일반적인 C# 조건문을 WPF 이벤트 처리기에 적용합니다.
- 빈 입력 안내를 포함한 1주차 프로그램을 완성합니다.

시작 상태는 3일차 완료본입니다. 오늘 새로 추가하는 것은 입력값 확인과 상태 문구뿐입니다. 남은 시간은 같은 프로그램을 실행하고 디버거로 값을 따라가는 데 사용합니다.

2. 핵심 개념 설명 — 15분

버튼 클릭 후 처리할 순서를 먼저 말로 정해 봅니다.

```
현재 입력값 읽기
  → 앞뒤 공백 제거
  → 내용이 비어 있으면 안내하고 종료
  → 내용이 있으면 결과와 상태 표시
```

이 순서를 C#으로 적으면 됩니다. 화면 프로그램이라고 해서 조건문이나 string 처리 방법이 달라지지 않습니다.

- Trim(): 읽어 온 문자열의 앞뒤 공백을 제거합니다. TextBox 자체를 수정하는 것은 아닙니다.
- string.IsNullOrWhiteSpace(input): 입력이 없거나 공백뿐인지 확인합니다.
- return: 현재 이벤트 처리기만 끝냅니다. 창이나 프로그램 전체를 닫지 않습니다.

예를 들어 입력창이 " 테스트 "이면 결과는 "테스트"가 됩니다. 입력창에는 원래 글자가 남습니다. 입력창이 공백뿐이면 결과 영역에는 결과가 없다는 안내를, 상태 문구에는 입력 요청을 표시합니다.

결과와 상태는 역할이 다릅니다. 결과는 사용자가 얻으려는 값이고, 상태는 "표시 완료", "입력 필요"와 같은 진행 안내입니다. 다음 주에 결과가 표로 바뀌어도 상태 문구는 그대로 활용합니다.

XAML의 안내 문구만 바꾸는 일과 C#의 처리 순서를 바꾸는 일은 구분해서 생각합니다. UI(User Interface)는 사용자가 조작하거나 보는 화면 부분을 뜻합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

XAML에는 상태용 TextBlock 하나를 추가합니다. C#에는 입력값을 변수에 저장하고, 빈 입력이면 안내하는 조건문을 추가합니다.

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6"
                    Click="RunButton_Click" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
            <TextBlock x:Name="StatusTextBlock"
                       Text="입력 후 실행 버튼을 누르세요."
                       Margin="0,12,0,0" TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void RunButton_Click(object sender, RoutedEventArgs e)
    {
        string input = InputTextBox.Text.Trim();

        if (string.IsNullOrWhiteSpace(input))
        {
            ResultTextBlock.Text = "표시할 결과가 없습니다.";
            StatusTextBlock.Text = "텍스트를 먼저 입력해 주세요.";
            return;
        }

        ResultTextBlock.Text = input;
        StatusTextBlock.Text = "입력한 텍스트를 표시했습니다.";
    }
}
```

처리기에서 화면 값을 직접 읽고 쓰는 부분은 세 컨트롤의 Text 속성입니다. 나머지는 익숙한 C# 문자열 처리와 조건문입니다. 줄마다 외우기보다 "읽기 → 확인 → 표시"를 기억하세요.

4. Visual Studio에서 따라 하는 실습 — 75분

파일 준비와 구현 — 25분

1. 실행을 종료하고 MainWindow 두 파일을 엽니다.
2. XAML 전체를 교체합니다. 아래쪽 StackPanel에 StatusTextBlock이 추가된 것을 확인합니다.
3. C# 전체를 교체합니다. 생성자는 유지하고 RunButton_Click의 내용이 확장된 것을 확인합니다.
4. 모두 저장하고 **빌드 → 솔루션 빌드**를 누릅니다. 오류가 있으면 오류 목록에서 첫 오류를 더블클릭합니다. 컨트롤 이름과 Click 처리기 이름부터 맞춥니다.
5. F5로 실행해 입력창과 버튼, 결과와 상태 문구가 보이는지 확인합니다.

입력에 따른 화면 동작 확인 — 20분

1. "WPF 테스트"를 입력하고 실행을 누릅니다. 결과는 "WPF 테스트", 상태는 "입력한 텍스트를 표시했습니다."가 됩니다.
2. 입력을 모두 지우고 실행합니다. 이전 결과 대신 "표시할 결과가 없습니다."가 나타나고 상태는 "텍스트를 먼저 입력해 주세요."가 됩니다.
3. 공백만 넣고 실행해 같은 안내가 나오는지 확인합니다.

4. 앞뒤에 공백이 있는 " 다시 시작 "을 입력하고 실행합니다. 결과는 "다시 시작"이고, 입력창의 앞뒤 공백은 남아 있습니다.
5. 긴 문장을 넣고 창 너비를 조절합니다. 결과가 다음 줄로 넘어가는 것을 봅니다. 화면을 꾸미기 위한 코드는 추가하지 않습니다.

C# 디버거로 실행 흐름 관찰 — 20분

1. 실행 중인 창을 닫습니다.
2. C#의 `string input = InputTextBox.Text.Trim();` 줄 왼쪽 여백을 클릭해 중단점을 표시합니다. 같은 줄에서 F9를 눌러도 됩니다.
3. F5로 실행하고 입력 후 버튼을 누릅니다. Visual Studio가 해당 줄에서 멈춥니다. 디버거가 멈춘 동안 앱이 반응하지 않는 것은 정상입니다.
4. F10을 한 번 눌러 대입문을 실행합니다. 그다음 input에 마우스를 올리거나 **디버그 → 창 → 지역**에서 값을 확인합니다. 노란 줄은 다음에 실행할 줄입니다.
5. F10으로 조건문과 결과 표시를 따라갑니다. F5를 누르면 계속 실행합니다.
6. 공백만 입력한 뒤 버튼을 다시 누릅니다. 이번에는 if 내부를 거쳐 return에서 처리기가 끝나는 것을 봅니다. F5로 계속 실행하면 창은 열린 채 유지됩니다.
7. 중단점을 다시 클릭하거나 F9로 제거하고 실행을 종료합니다.

최종 저장과 화면 확인 — 10분

1. 중단점이 제거된 상태에서 저장하고 F5로 다시 실행합니다.
2. 텍스트 입력 → 클릭 → 결과 표시까지 한 번 이어서 진행합니다.
3. MainWindow.xaml의 Click과 C# 메서드를 나란히 열고, 입력값을 읽는 줄과 결과를 쓰는 줄의 연결을 확인합니다.
4. 창을 닫습니다. 다음 주에는 이 프로젝트를 그대로 이어 씁니다.

시간이 부족하면 긴 문장의 창 너비 조절을 먼저 생략하고, 정상 입력과 빈 입력의 동작을 우선 확인합니다. 별도의 응용 프로그램은 만들지 않습니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- WPF의 화면 요소도 C#에서 속성을 읽고 쓸 수 있는 객체입니다.
- XAML은 배치와 초기 속성, Code-behind는 클릭 후 동작을 담당했습니다.
- 입력값 읽기, 문자열 처리, 조건문은 일반적인 C# 코드입니다.
- return은 클릭 처리기를 끝내며, 다음 클릭에는 같은 메서드가 다시 호출됩니다.
- 현재는 화면을 직접 읽고 수정합니다. 다음 주에는 C# 데이터 속성과 화면을 바인딩으로 연결합니다.

실무·면접 기초: "WPF에서 XAML과 C#은 어떻게 연결되는가?"에는 x:Class로 같은 창 클래스를 연결하고, InitializeComponent로 화면을 초기화하며, Click으로 이벤트 처리기를 연결한다고 설명할 수 있으면 충분합니다.

완료본은 day04 폴더입니다. 지금 학습 상태에는 API 호출이나 데이터 표가 아직 없습니다. 다음은 2주차 1일차입니다. 현재 제공 프로젝트는 전체 과정 완료 상태이므로, 1주차 완료 상태가 필요하면 단계 적용 안내에 따라 Stage 4를 적용합니다.